

This is a comprehensive overview of the core concepts, processes, pseudocode examples, and formulas covered in your Agentic AI course, drawn directly from the provided source material. Use this as a focused study guide for your exam.

I. Foundational Concepts

What is Agentic AI?

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines programmed to learn, reason, and make decisions. An **Agent** is an "intelligent" system that interacts with an "environment" by taking actions based on observations. AI Agents can perform specific tasks autonomously and may require oversight.

Agentic AI is a framework for goal-oriented behavior under limited supervision, built around autonomous AI agents. Unlike traditional automation, **Agentic AI** makes decisions and learns independently, handles ambiguity, and adapts in real-time.

Key Characteristics of Agentic AI:

- **Autonomous:** Agents operate independently, making decisions and taking actions without constant human intervention.
- **Goal-Oriented:** Systems are designed with specific goals in mind.
- **Self-Learning:** They continuously improve through interactions (Learning and Adaptation).
- **Contextual Awareness:** They can understand and adapt to changing contexts based on real-time information.

Agents vs. LLMs

The Agent and the Large Language Model (LLM) are not the same; the agent is considered an **instrument**. | Component | Function | | :--- | :--- | | **Agent** | Decides *when* to call the LLM and orchestrates the whole process. | | **LLM** | Chooses the tool and provides reasoning. |
AI agents are often described as **Engineering Wrappers** around AI models.

II. Agent Architecture, Components, and Memory

Core Agent Components

An agentic architecture enables intelligent behavior through several components:

1. **Perception Module:** Processes data from sensors or input sources (like cameras or microphones) to interpret the environment.
2. **Reasoning Engine:** Applies algorithms and inference models to analyze information and make decisions, often accessing structured knowledge bases.
3. **Action Module:** The response to the perception and reasoning functionalities, executing necessary actions.
4. **Memory Systems / Knowledge Base:** Used for context storage.
5. **Tool Integration:** The ability to leverage external tools.
6. **Planning Module:** Used for planning strategies and making decisions.

Reasoning and Intermediate Steps (Chain-of-Thought)

LLMs are probabilistic models. To solve complex tasks, reasoning is crucial.

- When provided with examples that include **intermediate steps**, LLMs respond with intermediate steps, regardless of whether it's training, fine-tuning, or prompting.
- When a step-by-step reasoning path is present, LLMs have much **higher confidence** in decoding the final answer compared to direct-answer decoding.

• **Task Decomposition** (like Chain-of-Thought or CoT) is a decomposition framework used to transform complex tasks into multiple manageable tasks, shedding light on the model's thinking process.

Key Reasoning Formula (Self-Consistency): To maximize the probability of the final answer given the problem, while incorporating the reasoning path, the approach is called **Self-Consistency**, which greatly improves step-by-step reasoning.

$$\arg \max_{\mathbb{P}}(\text{final answer} \mid \text{problem}) = \sum_{\text{reasoning path}} \mathbb{P}(\text{reasoning path, final answer} \mid \text{problem})$$

Memory in Agents

AI models are typically **stateless by design** and do not have inherent memory, treating every input independently unless memory is architected into the system.

Types of Memory:

1. Short-term Memory (In-context memory):

- A **raw log** of user-agent conversations, scoped to a single session or task.
- Includes the conversation so far, tools used, responses generated, and documents retrieved.

2. Long-term Memory (Vector DB):

- Lives across sessions, days, weeks—even forever.
- Helps agents remember who the user is, how they prefer to interact, what has already been done, and important past context.

◦ Types of Long-term Memory:

- **Semantic Memory:** Objective facts and info (e.g., "User speaks English").
- **Episodic Memory:** Past actions (e.g., "Agent already generated a summary yesterday").
- **Procedural Memory:** Subjective preferences (e.g., "Avoid passive voice").

Tools and Function Calling

Tools are **external capabilities** that the LLM can invoke, such as APIs, database queries, internal services, third-party systems, or internal functions written in code.

Tool Calling vs. Function Calling:

- **Function Calling:** Refers to the LLM generating structured calls to predefined functions **within the same runtime** (e.g., internal tasks like database lookups or calculations).
- **Tool Calling:** Enables LLMs to interact with **external APIs, services, or systems**, allowing them to access real-time data and perform specialized tasks beyond their intrinsic capabilities.

Example Tool Invocation (JSON structure): When an LLM invokes a tool, it generates a structured response indicating: which tool/function to use, what parameters to pass, and how those parameters should be formatted.

```
{
  "function": "weather_lookup",
  "parameters": {
    "location": "Boston",
```

```
"date": "10/01/2024"
}
}[61, 62]
```

Tool Definition Approaches:

1. **Docstring Approach (using a framework):** Crucial information (tool purpose, required/optional arguments, expected return values) is provided within the docstring of a Python function.
2. **Direct LLM Integration:** Tools are defined using a **specific JSON schema format** that the LLM APIs understand.

Example Function Tool (Python):

```
@function_tool
def calculate_compound_interest(principal: float, rate: float, time: int) -> float:
    """Calculate compound interest"""
    return principal * (1 + rate) ** time
```

This decorator marks the Python function as a "tool" usable by the agent, providing metadata like name, description, and input/output schema so the LLM can reason.

III. Agent Execution and the Agent Loop

The Agent Loop Process

The Agent Loop is the core process defining how an agent runs and typically involves understanding LLM calls, tool execution, and handoffs.

1. **LLM Call:** The agent processes input (current agent + input) and generates a response.
2. **Output Analysis:** The output is checked for:
 - A **Final Output** (ends the loop).
 - **Tool Calls** (requires tool execution).
 - **Handoffs** (requires switching to a new agent).
3. **Tool Execution / Handoff Processing:** Any necessary tool calls are run and results are appended, or the agent switches if a handoff occurs.
4. **Loop Continuation:** Repeat steps until the final output is generated or a maximum number of turns (`max_turns`) is reached.

Agent Execution Methods (The Runner)

When building an agent (LLM + tools + memory), you decide how it executes using three main methods provided by the `Runner` orchestrator:

1. `Runner.run()` (Asynchronous):

- **Mode:** Asynchronous (non-blocking).
- **Usage:** Best when the application needs to handle multiple requests simultaneously.
- **Flow:** Submits the task, returns control immediately to the program, and the result is collected later.

2. `Runner.run_sync()` (Synchronous):

- **Mode:** Synchronous (blocking).
- **Usage:** Best when running one agent call at a time.
- **Flow:** Submits the task, the program waits until the agent finishes, and returns the final response directly.

3. `Runner.run_streamed()` (Streaming):

- **Mode:** Streaming (real-time events).
- **Usage:** Best when you want to show responses live (e.g., chatbots).
- **Flow:** Partial tokens or tool call updates are streamed back as events, allowing the application to display or process them in real-time.

IV. Data Integration and Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG)

RAG is a system design where the model retrieves relevant information from your own data just before generating a response. It is essential because models can only process a limited amount of text, and including too much irrelevant information (noise) makes responses less focused and error-prone.

The RAG Process:

1. **Data:** Use internal content (PDFs, emails, notes, wikis).
2. **Chunking:** Break the data into smaller parts for better indexing.
3. **Prompt + Context (Retrieval Phase):** At query time, the system retrieves relevant pieces.
4. **LLM:** The model uses that retrieved context to generate a response.
5. **Output:** The result is based on your data, not just what the model intrinsically knows.

Agentic RAG

In agentic workflows, retrieval is part of a broader, dynamic reasoning loop.

- **Traditional RAG** involves one query, one retrieval, and one response, working well for standalone questions.
- **Agentic RAG** is a mechanism for handling dynamic workflows and acts as a bridge between reasoning and grounded enterprise knowledge. Agents **plan, retrieve, reflect, and retrieve again** as many times as needed throughout a task, not just once.

LlamaIndex

LlamaIndex is a data framework that acts as middleware between your data sources (documents, PDFs, APIs) and your LLMs/agents.

Advantages of LlamaIndex over Memory-Only RAG:

Feature	Memory-only RAG	With LlamaIndex
Long-term recall	Lost	Enabled (docstore)
Vector embeddings	Absent	Enabled
Multi-source data	Absent	Enabled
Agent integration	Manual	Built-in tools
Short-term context	Enabled	Enabled

LlamaIndex is essential in serious Agentic RAG setups because its **persistence + retrieval = orchestration**.

LlamaIndex Pseudocode Example (Integration with Agents):

- 1. Ingestion of Data:
- 2. Create Query Engine:
- 3. Agent Uses Query Engine as a Tool:
- 4. Ask Questions:

V. Agent Design Patterns and Workflows

Agent Architectures (Types of Agents)

Architecture Type	Key Characteristics	Strengths	Limitations	Example Use Cases
Reactive Agents	Stimulus-response model, no internal memory or planning.	Fast response times, simple design.	Lack adaptability, no long-term learning.	Spam filters, Roomba vacuum cleaners.
Deliberative Agents	Use environmental models to plan actions. Sense-plan-act cycle (Sensing, Knowledge updating, Planning/Reasoning, Plan execution).	Can reason and make informed decisions.	Slower response time due to computation.	Autonomous vehicles, AI-powered decision support.
Hybrid Agents	Combine reactive and deliberative approaches.	Balance speed and decision-making.	Higher complexity in design and maintenance.	Virtual assistants like Siri and Alexa.

Multi-Agent Systems	Multiple AI programs collaborate or compete to solve problems.	Distributed intelligence, collaboration	Coordination/communication overheads.	Traffic management, autonomous drone swarms.
----------------------------	--	---	---------------------------------------	--

Agentic Design Patterns

These patterns allow agents to execute complex tasks dynamically:

1. **Reflection Pattern:** The agent critiques itself, spots mistakes, identifies gaps, and iterates until it produces an improved final answer (self-reflection).
2. **Tool Use Pattern:** The agent decides when to use an external tool (API, calculator, DB) to gather information, marking the first step toward agentic orchestration.
3. **ReAct Pattern (Reason + Act):** Combines reflection and tool use. The agent reasons step by step (thoughts), then takes an action (tool use, answer, or query), and the loop continues until the task is done.
4. **Planning Pattern:** Instead of answering directly, the agent first creates a roadmap (sequence of steps) by subdividing tasks or outlining objectives, and then executes the plan step by step. Useful for complex, multi-step tasks.
5. **Belief Desire Intention (BDI) Pattern:** Inspired by cognitive agent theory. The agent's behavior is guided by **Beliefs** (what the agent knows about the world/context), **Desires** (goals/objectives it wants to achieve), and **Intentions** (concrete plans/actions it commits to).

LLM Workflows

Workflows are used for structured, predictable processes where LLMs and tools are orchestrated through predefined code paths.

1. **Prompt Chaining:** Breaks a task into sequential steps, each handled by a separate LLM call. Useful for stepwise refinement.
2. **Routing Workflow:** Classifies the input and directs it to specialized prompts, models, or tools. Ideal for multi-domain systems.
3. **Parallelisation Workflow:** Executes multiple subtasks concurrently to increase speed and diversity of outputs. Suitable for multi-agent workflows.
4. **Evaluator-Optimizer Workflow:** One LLM generates output, and another evaluates and refines it in a feedback loop. Effective for iterative improvement (e.g., code generation).

CrewAI Framework & Multi-Agent Coordination

CrewAI is an open-source framework used to build multi-agent systems, providing a pattern to put these systems together and offering tools for agents.

Building Blocks of CrewAI: Agents, Tasks, and Crews.

Agent Characteristics in CrewAI: Role Playing, Focus, Tools, Cooperation, Guardrails, and Memory. Role playing includes defining the agent's **Role, Goals, and Backstory**.

Agent Coordination Patterns in Multi-Agent Systems:

- **Flat Patterns (More Dynamic):** Agents talk to each other as peers ("no boss"). Used when tasks need creativity, debate, mutual evaluation, or when there is no single correct answer path.
- **Hierarchical Patterns (More Controllable):** An orchestrator agent delegates subtasks to others, seeing the full picture and controlling the flow. Used when tasks can be clearly decomposed or when tight control over known agent roles (summarizer, generator) is needed.

CrewAI Collaboration Tools (When `allow_delegation=True`):

1. **Delegate Work Tool:** Allows agents to assign tasks to teammates with specific expertise.
2. **Ask Question Tool:** Enables agents to ask specific questions to gather information from colleagues.

Critique Loops (A challenge in Multi-Agent Systems): Critique loops occur when agents assigned evaluation tasks repeatedly review and improve the same content in a cycle without finalizing or terminating the loop.

How to Avoid Critique Loops:

1. Use `allow_delegation=False` for critic agents.
2. Define clear `expected_output` in the task to signal "done".
3. **Switch process type:** Use sequential instead of hierarchical re-delegation if strict flow is needed.
4. Add `max_iterations` (supported by CrewAI versions).
5. Consider implementing a **final judge agent** rather than relying on cyclic critique.

LangGraph

LangGraph is an advanced library built on LangChain designed to enable **cyclic computational capabilities** in LLM applications, which are often needed for agent runtimes. Unlike systems restricted to Directed Acyclic Graphs (DAGs) for linear workflows, LangGraph adds cycles, allowing LLMs to continuously loop through a process and dynamically decide the next action based on evolving conditions.

LangGraph Terminologies:

1. **Graphs:** Model agent workflows.
2. **State:** A shared data structure (typically a Type Dict or Pydantic Base Model) representing the current snapshot of the application.
3. **Nodes:** Python functions that contain the logic of your agents. They receive the current State as input, perform computation, and return an updated State.
4. **Edges:** Python functions that determine which Node to execute next based on the current State. They can be conditional branches or fixed transitions. Key types include **Normal Edges** (go directly to the next node) and **Conditional Edges** (call a function to decide the next node).
5. **Checkpoint:** A built-in persistence layer that saves a snapshot of the graph state at every super-step, enabling powerful capabilities like persistence for long-running applications.

VI. Safety and Protocols

Guardrails and Validation

Guardrails are automated safety mechanisms that validate inputs and outputs to ensure AI agents operate within acceptable boundaries, acting as safety checkpoints.

Purpose of Guardrails:

- Prevent processing of inappropriate or harmful content.
- Block responses that violate safety policies.
- Validate inputs against business rules and constraints.

Types of Guardrails:

1. **Input Guardrails:** Validate and filter user inputs before processing.
2. **Output Guardrails:** Check and sanitize agent responses before delivery.
3. **Guardrail Agents:** Specialized agents for validation and safety checks.

Example Guardrail Patterns (Pseudocode/Syntax):

- **Input Validation Pattern (using decorator):**
- **Business Rule Validation:**

Model Context Protocol (MCP)

MCP is an open standard designed to unify communication between LLMs and external data sources and tools, standardizing interactions and addressing limitations of traditional function calling methods. It serves as a "USB-C port" for AI agents, providing a universal interface for diverse components.

MCP Architecture Components:

- **MCP Server:** Operates independently, managing tools and resources.
- **LLM Client:** Connects to the MCP server to access tools.
- **MCP Hosts:** Programs (like Claude Desktop or IDEs) that access data through MCP.
- **Local Data Sources:** Your computer's files, databases, and services that MCP servers can securely access.
- **Remote Services:** External systems available over the internet (e.g., via APIs) that MCP servers can connect to.

Key Features of MCP:

- **Decoupling:** Separates tool management from the agent, enhancing modularity.
- **Reusability:** Tools can be reused across different agents and systems.
- **Scalability:** Facilitates the addition of new tools without modifying the core agent.
- **Communication:** Utilizes JSON-RPC 2.0 over standard I/O or Server-Sent Events (SSE).

Agent2Agent

Agent2Agent is a collaborative AI communication framework designed to simulate human-like interactions, supporting multi-agent coordination, negotiation, and task-solving through natural language. It is a step toward interactive, dynamic, and scalable AI ecosystems.

frame the scenario based questions...give their answers along with the questions

This response frames several key concepts from the Agentic AI curriculum as scenario-based questions, complete with answers directly supported by the provided material, to help you prepare for your exam.

Scenario-Based Questions for Agentic AI

Scenario 1: Agent Execution Modes

Question: A critical enterprise application is designed to handle thousands of concurrent customer requests, each requiring an autonomous AI agent to access external APIs. You are implementing the **Runner** component. Which execution method must you choose to ensure the application remains

responsive and handles multiple user requests simultaneously, rather than freezing while waiting for a single task to finish?

Answer:

The developer must choose the `Runner.run()` execution method.

- **Method:** `Runner.run()`

- **Mode:** **Asynchronous (non-blocking).**

- **Usage:** This is best when the application needs to handle multiple requests simultaneously.

- **Flow:** It submits the task and immediately returns control to the program, allowing the program to continue running other tasks while the agent result is awaited or collected later.

(In contrast, `Runner.run_sync()` would be chosen if you were only running one agent call at a time, as it is synchronous and blocking.)

Scenario 2: Memory Management for Personalized Agents

Question: An Agentic AI system is used as a highly personalized virtual assistant that manages a user's calendar, drafts emails, and handles long-term projects over several weeks. Identify the two types of **Long-Term Memory** this agent needs to implement if it must: 1) recall that the user prefers all communication to "Avoid passive voice," and 2) remember that the agent previously "generated a summary yesterday."

Answer:

The agent needs to utilize two specific types of Long-Term Memory:

1. **Procedural Memory (Subjective Preferences):** This memory stores subjective preferences. It would be used to store the preference: **"Avoid passive voice. Prioritize action items."**

2. **Episodic Memory (Past Actions):** This memory records past actions. It would be used to store the historical action: **"Agent already generated a summary yesterday."**

(Long-term memory lives across sessions, days, weeks, or even forever, helping the agent remember important past context and how the user prefers to interact.)

Scenario 3: Task Decomposition and Reasoning

Question: An LLM is tasked with solving a complex math problem: *Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for \$2 per egg. How much money does she make every day?*

If the model initially fails to provide the correct answer, what crucial element must be added to the input prompt (regardless of whether you are training, fine-tuning, or prompting) to improve accuracy and ensure the correct output is decoded reliably? How does the **Self-Consistency** technique maximize the final probability of the correct answer?

Answer:

1. **Crucial Element to Add:** The system must be provided with demonstration examples that include **intermediate steps**, which trigger the **reasoning process** before the final answer. The model must be instructed to "derive the final answer through a series of small steps". When a step-by-step reasoning path is present, LLMs have much higher confidence in decoding the final answer.

- *Example of Intermediate Step:* "She eats 3 for breakfast, so she has $16 - 3 = 13$ left. Then she bakes muffins, so she has $13 - 4 = 9$ eggs left. She sells $9 \text{ eggs} * \$2 = \18 ."

2. **Self-Consistency:** Self-Consistency is the technique that greatly improves step-by-step reasoning. It maximizes the probability of the final answer given the problem by summing over all possible reasoning paths that lead to that answer.

- 3.

$$\text{argmax}P(\text{final answer} \mid \text{problem})=$$

reasoning path

$$\Sigma$$

$P(\text{reasoning path, final answer} \mid \text{problem})$

Scenario 4: Multi-Agent System Design

Question: You are building a CrewAI system where a specialized Data Analyst agent calculates metrics, and a Content Lead agent synthesizes the report. The Content Lead needs the ability to break down the final writing task and delegate subtasks to the Data Analyst.

1. What coordination pattern does this structure (one agent delegating to others) represent?
2. What specific parameter setting must be enabled for the Content Lead agent in CrewAI to allow this delegation and utilize the built-in collaboration tools?
3. What specific practice should be enforced for the Data Analyst if you want it to focus purely on calculations and avoid getting distracted by complex collaboration requests?

Answer:

1. **Coordination Pattern:** This represents a **Hierarchical Pattern**. An orchestrator agent (the Content Lead) delegates subtasks to others (the Data Analyst). This pattern is used when tasks can be clearly decomposed and when you need tight control or have known agent roles (e.g., summarizer, checker).
2. **Delegation Setting:** The parameter `allow_delegation` must be set to `True` for the Content Lead agent. When this is enabled, CrewAI automatically provides two powerful tools: the **Delegate Work Tool** (to assign tasks to teammates) and the **Ask Question Tool** (to gather information from colleagues).
3. **Data Analyst Practice:** To ensure the Data Analyst focuses solely on its core expertise and avoids delegation overhead, the configuration for the specialist agent should set `allow_delegation` to `False`.

Scenario 5: Guardrails and Safety

Question: You are setting up security for a financial AI agent. The agent receives user input (e.g., "Transfer \$10,000 to Account X") and generates an output (e.g., "Transaction initiated").

1. Explain what a **Guardrail** is, and state which type of guardrail pattern must be implemented to prevent a user from submitting an input containing inappropriate or harmful content.
2. If the input passes the safety checks but the agent hallucinates and generates an unsafe response, which guardrail pattern handles this violation before the response reaches the user?

Answer:

1. Guardrail Definition and Input Filtering:

- **Definition:** Guardrails are **automated safety mechanisms** that act as **safety checkpoints**. Their purpose is to validate inputs and outputs to ensure AI agents operate within acceptable boundaries. They help prevent the processing of inappropriate or harmful content.

- **Pattern:** To filter the user's input before processing, you need **Input Guardrails**. This can be implemented using the **Input Validation Pattern**, potentially including a Business Rule Validation check (e.g., ensuring the transfer amount is within a set limit).

2. **Output Filtering:** To block an unsafe or hallucinated response generated by the agent before it reaches the user, you must implement **Output Guardrails**. This uses the **Output Safety Pattern**. If

a safety check is triggered, the **Exception Handling Pattern** can be used to return a message like "Response blocked for safety reasons".

Scenario 6: RAG vs. Agentic RAG

Question: A company has hundreds of internal PDFs and documents.

1. Why is it generally problematic to give *all* this data directly to an LLM without using Retrieval-Augmented Generation (RAG)?
2. If the task is simply "Find the policy regarding vacation days," is traditional RAG or Agentic RAG sufficient?
3. If the task is changed to: "Develop a plan to overhaul the vacation policy, using retrieved documents, then critique the resulting plan, and finally generate a formal memo," why does this task now require **Agentic RAG** specifically?

Answer:

1. **Problem with Direct Input:** Models can only process a **limited amount of text at a time**. Furthermore, even within that limit, they struggle when **too much irrelevant or noisy information is included**, which makes responses less focused and more error-prone. RAG solves this by retrieving only the relevant information.
2. **Simple Task:** Traditional RAG is sufficient for this standalone question. Traditional RAG generally consists of one query, one retrieval, and one response.
3. **Complex Task Requirement (Agentic RAG):** The complex, multi-step nature of the overhaul task requires **Agentic RAG**. Agentic RAG is necessary because it treats retrieval as part of a dynamic reasoning loop. The agents must actively **plan, retrieve, reflect, and retrieve again**—not just once, but multiple times as needed throughout the completion of the task. Agentic RAG is a mechanism for handling dynamic workflows and acts as a bridge between reasoning and grounded enterprise knowledge.
covered in your Agentic AI course, drawn directly from the provided source material. Use this as a focused study guide for your exam.

I. Foundational Concepts

What is Agentic AI?

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines programmed to learn, reason, and make decisions. An **Agent** is an "intelligent" system that interacts with an "environment" by taking actions based on observations. AI Agents can perform specific tasks autonomously and may require oversight.

Agentic AI is a framework for goal-oriented behavior under limited supervision, built around autonomous AI agents. Unlike traditional automation, **Agentic AI** makes decisions and learns independently, handles ambiguity, and adapts in real-time.

Key Characteristics of Agentic AI:

- **Autonomous:** Agents operate independently, making decisions and taking actions without constant human intervention.
- **Goal-Oriented:** Systems are designed with specific goals in mind.
- **Self-Learning:** They continuously improve through interactions (Learning and Adaptation).
- **Contextual Awareness:** They can understand and adapt to changing contexts based on real-time information.

Agents vs. LLMs

The Agent and the Large Language Model (LLM) are not the same; the agent is considered an **instrument**. | Component | Function | | :--- | :--- | | **Agent** | Decides *when* to call the LLM and orchestrates the whole process. | | **LLM** | Chooses the tool and provides reasoning. |
AI agents are often described as **Engineering Wrappers** around AI models.

II. Agent Architecture, Components, and Memory

Core Agent Components

An agentic architecture enables intelligent behavior through several components:

1. **Perception Module:** Processes data from sensors or input sources (like cameras or microphones) to interpret the environment.
2. **Reasoning Engine:** Applies algorithms and inference models to analyze information and make decisions, often accessing structured knowledge bases.
3. **Action Module:** The response to the perception and reasoning functionalities, executing necessary actions.
4. **Memory Systems / Knowledge Base:** Used for context storage.
5. **Tool Integration:** The ability to leverage external tools.
6. **Planning Module:** Used for planning strategies and making decisions.

Reasoning and Intermediate Steps (Chain-of-Thought)

LLMs are probabilistic models. To solve complex tasks, reasoning is crucial.

- When provided with examples that include **intermediate steps**, LLMs respond with intermediate steps, regardless of whether it's training, fine-tuning, or prompting.
- When a step-by-step reasoning path is present, LLMs have much **higher confidence** in decoding the final answer compared to direct-answer decoding.
- **Task Decomposition** (like Chain-of-Thought or CoT) is a decomposition framework used to transform complex tasks into multiple manageable tasks, shedding light on the model's thinking process.

Key Reasoning Formula (Self-Consistency): To maximize the probability of the final answer given the problem, while incorporating the reasoning path, the approach is called **Self-Consistency**, which greatly improves step-by-step reasoning.

$$\arg \max_{\mathbb{P}}(\text{final answer} \mid \text{problem}) = \sum_{\text{reasoning path}} \mathbb{P}(\text{reasoning path, final answer} \mid \text{problem})$$

Memory in Agents

AI models are typically **stateless by design** and do not have inherent memory, treating every input independently unless memory is architected into the system.

Types of Memory:

1. **Short-term Memory (In-context memory):**
 - A **raw log** of user-agent conversations, scoped to a single session or task.
 - Includes the conversation so far, tools used, responses generated, and documents retrieved.
2. **Long-term Memory (Vector DB):**
 - Lives across sessions, days, weeks—even forever.
 - Helps agents remember who the user is, how they prefer to interact, what has already been done, and important past context.
 - **Types of Long-term Memory:**
 - **Semantic Memory:** Objective facts and info (e.g., "User speaks English").
 - **Episodic Memory:** Past actions (e.g., "Agent already generated a summary yesterday").
 - **Procedural Memory:** Subjective preferences (e.g., "Avoid passive voice").

Tools and Function Calling

Tools are **external capabilities** that the LLM can invoke, such as APIs, database queries, internal services, third-party systems, or internal functions written in code.

Tool Calling vs. Function Calling:

- **Function Calling:** Refers to the LLM generating structured calls to predefined functions **within the same runtime** (e.g., internal tasks like database lookups or calculations).
- **Tool Calling:** Enables LLMs to interact with **external APIs, services, or systems**, allowing them to access real-time data and perform specialized tasks beyond their intrinsic capabilities.

Example Tool Invocation (JSON structure): When an LLM invokes a tool, it generates a structured response indicating: which tool/function to use, what parameters to pass, and how those parameters should be formatted.

```
{
  "function": "weather_lookup",
  "parameters": {
    "location": "Boston",
    "date": "10/01/2024"
  }
} [61, 62]
```

Tool Definition Approaches:

1. **Docstring Approach (using a framework):** Crucial information (tool purpose, required/optional arguments, expected return values) is provided within the docstring of a Python function.
2. **Direct LLM Integration:** Tools are defined using a **specific JSON schema format** that the LLM APIs understand.

Example Function Tool (Python):

```
@function_tool
def calculate_compound_interest(principal: float, rate: float, time: int) -> float:
    """Calculate compound interest"""
    return principal * (1 + rate) ** time
```

This decorator marks the Python function as a "tool" usable by the agent, providing metadata like name, description, and input/output schema so the LLM can reason.

III. Agent Execution and the Agent Loop

The Agent Loop Process

The Agent Loop is the core process defining how an agent runs and typically involves understanding LLM calls, tool execution, and handoffs.

1. **LLM Call:** The agent processes input (current agent + input) and generates a response.
2. **Output Analysis:** The output is checked for:
 - A **Final Output** (ends the loop).
 - **Tool Calls** (requires tool execution).
 - **Handoffs** (requires switching to a new agent).
3. **Tool Execution / Handoff Processing:** Any necessary tool calls are run and results are appended, or the agent switches if a handoff occurs.
4. **Loop Continuation:** Repeat steps until the final output is generated or a maximum number of turns (`max_turns`) is reached.

Agent Execution Methods (The Runner)

When building an agent (LLM + tools + memory), you decide how it executes using three main methods provided by the `Runner` orchestrator:

1. `Runner.run()` (**Asynchronous**):

- **Mode:** Asynchronous (non-blocking).
- **Usage:** Best when the application needs to handle multiple requests simultaneously.
- **Flow:** Submits the task, returns control immediately to the program, and the result is collected later.

2. `Runner.run_sync()` (**Synchronous**):

- **Mode:** Synchronous (blocking).
- **Usage:** Best when running one agent call at a time.
- **Flow:** Submits the task, the program waits until the agent finishes, and returns the final response directly.

3. `Runner.run_streamed()` (**Streaming**):

- **Mode:** Streaming (real-time events).
- **Usage:** Best when you want to show responses live (e.g., chatbots).
- **Flow:** Partial tokens or tool call updates are streamed back as events, allowing the application to display or process them in real-time.

IV. Data Integration and Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG)

RAG is a system design where the model retrieves relevant information from your own data just before generating a response. It is essential because models can only process a limited amount of text, and including too much irrelevant information (noise) makes responses less focused and error-prone.

The RAG Process:

1. **Data:** Use internal content (PDFs, emails, notes, wikis).
2. **Chunking:** Break the data into smaller parts for better indexing.
3. **Prompt + Context (Retrieval Phase):** At query time, the system retrieves relevant pieces.
4. **LLM:** The model uses that retrieved context to generate a response.
5. **Output:** The result is based on your data, not just what the model intrinsically knows.

Agentic RAG

In agentic workflows, retrieval is part of a broader, dynamic reasoning loop.

- **Traditional RAG** involves one query, one retrieval, and one response, working well for standalone questions.

- **Agentic RAG** is a mechanism for handling dynamic workflows and acts as a bridge between reasoning and grounded enterprise knowledge. Agents **plan, retrieve, reflect, and retrieve again** as many times as needed throughout a task, not just once.

LlamaIndex

LlamaIndex is a data framework that acts as middleware between your data sources (documents, PDFs, APIs) and your LLMs/agents.

Advantages of LlamaIndex over Memory-Only RAG:

Feature	Memory-only RAG	With LlamaIndex
Long-term recall	Lost	Enabled (docstore)
Vector embeddings	Absent	Enabled
Multi-source data	Absent	Enabled
Agent integration	Manual	Built-in tools
Short-term context	Enabled	Enabled

LlamaIndex is essential in serious Agentic RAG setups because its **persistence + retrieval = orchestration**.

LlamaIndex Pseudocode Example (Integration with Agents):

1. **Ingestion of Data:**
2. **Create Query Engine:**
3. **Agent Uses Query Engine as a Tool:**
4. **Ask Questions:**

V. Agent Design Patterns and Workflows

Agent Architectures (Types of Agents)

Architecture Type	Key Characteristics	Strengths	Limitations	Example Use Cases
Reactive Agents	Stimulus-response model, no internal memory or planning.	Fast response times, simple design.	Lack adaptability, no long-term learning.	Spam filters, Roomba vacuum cleaners.
Deliberative Agents	Use environmental models to plan actions. Sense-plan-act cycle (Sensing, Knowledge updating, Planning/Reasoning, Plan execution).	Can reason and make informed decisions.	Slower response time due to computation.	Autonomous vehicles, AI-powered decision support.
Hybrid Agents	Combine reactive and deliberative approaches.	Balance speed and decision-making.	Higher complexity in design and maintenance.	Virtual assistants like Siri and Alexa.
Multi-Agent Systems	Multiple AI programs collaborate or compete to solve problems.	Distributed intelligence, collaboration.	Coordination/communication overheads.	Traffic management, autonomous drone swarms.

Agentic Design Patterns

These patterns allow agents to execute complex tasks dynamically:

1. **Reflection Pattern:** The agent critiques itself, spots mistakes, identifies gaps, and iterates until it produces an improved final answer (self-reflection).
2. **Tool Use Pattern:** The agent decides when to use an external tool (API, calculator, DB) to gather information, marking the first step toward agentic orchestration.
3. **ReAct Pattern (Reason + Act):** Combines reflection and tool use. The agent reasons step by step (thoughts), then takes an action (tool use, answer, or query), and the loop continues until the task is done.
4. **Planning Pattern:** Instead of answering directly, the agent first creates a roadmap (sequence of steps) by subdividing tasks or outlining objectives, and then executes the plan step by step. Useful for complex, multi-step tasks.
5. **Belief Desire Intention (BDI) Pattern:** Inspired by cognitive agent theory. The agent's behavior is guided by **Beliefs** (what the agent knows about the world/context), **Desires** (goals/objectives it wants to achieve), and **Intentions** (concrete plans/actions it commits to).

LLM Workflows

Workflows are used for structured, predictable processes where LLMs and tools are orchestrated through predefined code paths.

1. **Prompt Chaining:** Breaks a task into sequential steps, each handled by a separate LLM call. Useful for stepwise refinement.
2. **Routing Workflow:** Classifies the input and directs it to specialized prompts, models, or tools. Ideal for multi-domain systems.
3. **Parallelisation Workflow:** Executes multiple subtasks concurrently to increase speed and diversity of outputs. Suitable for multi-agent workflows.
4. **Evaluator-Optimizer Workflow:** One LLM generates output, and another evaluates and refines it in a feedback loop. Effective for iterative improvement (e.g., code generation).

CrewAI Framework & Multi-Agent Coordination

CrewAI is an open-source framework used to build multi-agent systems, providing a pattern to put these systems together and offering tools for agents.

Building Blocks of CrewAI: Agents, Tasks, and Crews.

Agent Characteristics in CrewAI: Role Playing, Focus, Tools, Cooperation, Guardrails, and Memory. Role playing includes defining the agent's **Role, Goals, and Backstory**.

Agent Coordination Patterns in Multi-Agent Systems:

- **Flat Patterns (More Dynamic):** Agents talk to each other as peers ("no boss"). Used when tasks need creativity, debate, mutual evaluation, or when there is no single correct answer path.
- **Hierarchical Patterns (More Controllable):** An orchestrator agent delegates subtasks to others, seeing the full picture and controlling the flow. Used when tasks can be clearly decomposed or when tight control over known agent roles (summarizer, generator) is needed.

CrewAI Collaboration Tools (When `allow_delegation=True`):

1. **Delegate Work Tool:** Allows agents to assign tasks to teammates with specific expertise.
2. **Ask Question Tool:** Enables agents to ask specific questions to gather information from colleagues.

Critique Loops (A challenge in Multi-Agent Systems): Critique loops occur when agents assigned evaluation tasks repeatedly review and improve the same content in a cycle without finalizing or terminating the loop.

How to Avoid Critique Loops:

1. Use `allow_delegation=False` for critic agents.
2. Define clear `expected_output` in the task to signal "done".

3. **Switch process type:** Use sequential instead of hierarchical re-delegation if strict flow is needed.
4. **Add `max_iterations`** (supported by CrewAI versions).
5. Consider implementing a **final judge agent** rather than relying on cyclic critique.

LangGraph

LangGraph is an advanced library built on LangChain designed to enable **cyclic computational capabilities** in LLM applications, which are often needed for agent runtimes. Unlike systems restricted to Directed Acyclic Graphs (DAGs) for linear workflows, LangGraph adds cycles, allowing LLMs to continuously loop through a process and dynamically decide the next action based on evolving conditions.

LangGraph Terminologies:

1. **Graphs:** Model agent workflows.
2. **State:** A shared data structure (typically a Type Dict or Pydantic Base Model) representing the current snapshot of the application.
3. **Nodes:** Python functions that contain the logic of your agents. They receive the current State as input, perform computation, and return an updated State.
4. **Edges:** Python functions that determine which Node to execute next based on the current State. They can be conditional branches or fixed transitions. Key types include **Normal Edges** (go directly to the next node) and **Conditional Edges** (call a function to decide the next node).
5. **Checkpoint:** A built-in persistence layer that saves a snapshot of the graph state at every super-step, enabling powerful capabilities like persistence for long-running applications.

VI. Safety and Protocols

Guardrails and Validation

Guardrails are automated safety mechanisms that validate inputs and outputs to ensure AI agents operate within acceptable boundaries, acting as safety checkpoints.

Purpose of Guardrails:

- Prevent processing of inappropriate or harmful content.
- Block responses that violate safety policies.
- Validate inputs against business rules and constraints.

Types of Guardrails:

1. **Input Guardrails:** Validate and filter user inputs before processing.
2. **Output Guardrails:** Check and sanitize agent responses before delivery.
3. **Guardrail Agents:** Specialized agents for validation and safety checks.

Example Guardrail Patterns (Pseudocode/Syntax):

- **Input Validation Pattern (using decorator):**
- **Business Rule Validation:**

Model Context Protocol (MCP)

MCP is an open standard designed to unify communication between LLMs and external data sources and tools, standardizing interactions and addressing limitations of traditional function calling methods. It serves as a "USB-C port" for AI agents, providing a universal interface for diverse components.

MCP Architecture Components:

- **MCP Server:** Operates independently, managing tools and resources.
- **LLM Client:** Connects to the MCP server to access tools.
- **MCP Hosts:** Programs (like Claude Desktop or IDEs) that access data through MCP.
- **Local Data Sources:** Your computer's files, databases, and services that MCP servers can securely access.

- **Remote Services:** External systems available over the internet (e.g., via APIs) that MCP servers can connect to.

Key Features of MCP:

- **Decoupling:** Separates tool management from the agent, enhancing modularity.
- **Reusability:** Tools can be reused across different agents and systems.
- **Scalability:** Facilitates the addition of new tools without modifying the core agent.
- **Communication:** Utilizes JSON-RPC 2.0 over standard I/O or Server-Sent Events (SSE).

Agent2Agent

Agent2Agent is a collaborative AI communication framework designed to simulate human-like interactions, supporting multi-agent coordination, negotiation, and task-solving through natural language. It is a step toward interactive, dynamic, and scalable AI ecosystems.

frame the scenario based questions...give their answers along with the questions

This response frames several key concepts from the Agentic AI curriculum as scenario-based questions, complete with answers directly supported by the provided material, to help you prepare for your exam.

Scenario-Based Questions for Agentic AI

Scenario 1: Agent Execution Modes

Question: A critical enterprise application is designed to handle thousands of concurrent customer requests, each requiring an autonomous AI agent to access external APIs. You are implementing the **Runner** component. Which execution method must you choose to ensure the application remains responsive and handles multiple user requests simultaneously, rather than freezing while waiting for a single task to finish?

Answer:

The developer must choose the **`Runner.run()`** execution method.

- **Method:** **`Runner.run()`**
- **Mode:** **Asynchronous (non-blocking).**
- **Usage:** This is best when the application needs to handle multiple requests simultaneously.
- **Flow:** It submits the task and immediately returns control to the program, allowing the program to continue running other tasks while the agent result is awaited or collected later.
(In contrast, `Runner.run_sync()` would be chosen if you were only running one agent call at a time, as it is synchronous and blocking.)

Scenario 2: Memory Management for Personalized Agents

Question: An Agentic AI system is used as a highly personalized virtual assistant that manages a user's calendar, drafts emails, and handles long-term projects over several weeks. Identify the two types of **Long-Term Memory** this agent needs to implement if it must: 1) recall that the user prefers all communication to "Avoid passive voice," and 2) remember that the agent previously "generated a summary yesterday."

Answer:

The agent needs to utilize two specific types of Long-Term Memory:

1. **Procedural Memory (Subjective Preferences):** This memory stores subjective preferences. It would be used to store the preference: **"Avoid passive voice. Prioritize action items."**
2. **Episodic Memory (Past Actions):** This memory records past actions. It would be used to store the historical action: **"Agent already generated a summary yesterday."**

(Long-term memory lives across sessions, days, weeks, or even forever, helping the agent remember important past context and how the user prefers to interact.)

Scenario 3: Task Decomposition and Reasoning

Question: An LLM is tasked with solving a complex math problem: *Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for \$2 per egg. How much money does she make every day?*

If the model initially fails to provide the correct answer, what crucial element must be added to the input prompt (regardless of whether you are training, fine-tuning, or prompting) to improve accuracy and ensure the correct output is decoded reliably? How does the **Self-Consistency** technique maximize the final probability of the correct answer?

Answer:

1. **Crucial Element to Add:** The system must be provided with demonstration examples that include **intermediate steps**, which trigger the **reasoning process** before the final answer. The model must be instructed to "derive the final answer through a series of small steps". When a step-by-step reasoning path is present, LLMs have much higher confidence in decoding the final answer.

◦ *Example of Intermediate Step:* "She eats 3 for breakfast, so she has 16 - 3 = 13 left. Then she bakes muffins, so she has 13 - 4 = 9 eggs left. She sells 9 eggs * \$2 = \$18."

2. **Self-Consistency:** Self-Consistency is the technique that greatly improves step-by-step reasoning. It maximizes the probability of the final answer given the problem by summing over all possible reasoning paths that lead to that answer.

3.

$$\operatorname{argmax}_{\text{reasoning path}} P(\text{final answer} \mid \text{problem}) =$$

reasoning path

$$\sum$$

$$P(\text{reasoning path, final answer} \mid \text{problem})$$

Scenario 4: Multi-Agent System Design

Question: You are building a CrewAI system where a specialized Data Analyst agent calculates metrics, and a Content Lead agent synthesizes the report. The Content Lead needs the ability to break down the final writing task and delegate subtasks to the Data Analyst.

1. What coordination pattern does this structure (one agent delegating to others) represent?
2. What specific parameter setting must be enabled for the Content Lead agent in CrewAI to allow this delegation and utilize the built-in collaboration tools?
3. What specific practice should be enforced for the Data Analyst if you want it to focus purely on calculations and avoid getting distracted by complex collaboration requests?

Answer:

1. **Coordination Pattern:** This represents a **Hierarchical Pattern**. An orchestrator agent (the Content Lead) delegates subtasks to others (the Data Analyst). This pattern is used when tasks can be clearly decomposed and when you need tight control or have known agent roles (e.g., summarizer, checker).

2. **Delegation Setting:** The parameter `allow_delegation` must be set to `True` for the Content Lead agent. When this is enabled, CrewAI automatically provides two powerful tools: the **Delegate Work Tool** (to assign tasks to teammates) and the **Ask Question Tool** (to gather information from colleagues).

3. **Data Analyst Practice:** To ensure the Data Analyst focuses solely on its core expertise and avoids delegation overhead, the configuration for the specialist agent should set `allow_delegation` to `False`.

Scenario 5: Guardrails and Safety

Question: You are setting up security for a financial AI agent. The agent receives user input (e.g., "Transfer \$10,000 to Account X") and generates an output (e.g., "Transaction initiated").

1. Explain what a **Guardrail** is, and state which type of guardrail pattern must be implemented to prevent a user from submitting an input containing inappropriate or harmful content.
2. If the input passes the safety checks but the agent hallucinates and generates an unsafe response, which guardrail pattern handles this violation before the response reaches the user?

Answer:

1. **Guardrail Definition and Input Filtering:**

- **Definition:** Guardrails are **automated safety mechanisms** that act as **safety checkpoints**. Their purpose is to validate inputs and outputs to ensure AI agents operate within acceptable boundaries. They help prevent the processing of inappropriate or harmful content.

- **Pattern:** To filter the user's input before processing, you need **Input Guardrails**. This can be implemented using the **Input Validation Pattern**, potentially including a Business Rule Validation check (e.g., ensuring the transfer amount is within a set limit).

2. **Output Filtering:** To block an unsafe or hallucinated response generated by the agent before it reaches the user, you must implement **Output Guardrails**. This uses the **Output Safety Pattern**. If a safety check is triggered, the **Exception Handling Pattern** can be used to return a message like "Response blocked for safety reasons".

Scenario 6: RAG vs. Agentic RAG

Question: A company has hundreds of internal PDFs and documents.

1. Why is it generally problematic to give *all* this data directly to an LLM without using Retrieval-Augmented Generation (RAG)?
2. If the task is simply "Find the policy regarding vacation days," is traditional RAG or Agentic RAG sufficient?
3. If the task is changed to: "Develop a plan to overhaul the vacation policy, using retrieved documents, then critique the resulting plan, and finally generate a formal memo," why does this task now require **Agentic RAG** specifically?

Answer:

1. **Problem with Direct Input:** Models can only process a **limited amount of text at a time**.

Furthermore, even within that limit, they struggle when **too much irrelevant or noisy information is included**, which makes responses less focused and more error-prone. RAG solves this by retrieving only the relevant information.

2. **Simple Task:** Traditional RAG is sufficient for this standalone question. Traditional RAG generally consists of one query, one retrieval, and one response.

3. **Complex Task Requirement (Agentic RAG):** The complex, multi-step nature of the overhaul task requires **Agentic RAG**. Agentic RAG is necessary because it treats retrieval as part of a dynamic reasoning loop. The agents must actively **plan, retrieve, reflect, and retrieve again**—not just once, but multiple times as needed throughout the completion of the task. Agentic RAG is a mechanism for handling dynamic workflows and acts as a bridge between reasoning and grounded enterprise knowledge.